Lecture 1. Computer Arithmetic

Integers, floating-point numbers, rounding, machine epsilon, error analysis, cancellation

Contents

1.1 Storage units and characters **2**

1.2 Integers: fixed-point numbers **2**

1.3 Floating-point numbers **2**

1.4 Rounding and machine epsilon **3**

1.5 Floating-point arithmetic and its pitfalls **4**

1.6 Selected exercises **6**

1.1 Storage units and characters

A **bit** (binary digit, a word coined by Tukey) takes the values 0 and 1; a **byte** is 8 bits. Decimal prefixes (KB $= 10^3$, MB $= 10^6$, GB $= 10^9$, TB $= 10^{12}$ bytes, ...) are distinguished from binary ones (KiB $= 2^{10}$, MiB $= 2^{20}$ bytes, ...).

Text is stored as character codes. ASCII uses 7 bits (128 characters) and extended ASCII 8 bits (256). Unicode encodings (UTF-8, UTF-16, UTF-32) cover all scripts; UTF-8, whose one-byte codes agree with ASCII, is dominant on the internet.

Memory is finite, so *every* representation of $\mathbb{Z}$ or $\mathbb{R}$ in a computer is an approximation.

1.2 Integers: fixed-point numbers

A fixed-point system with M bits models integers.

- **Unsigned** integers represent $\{0, 1, \ldots, 2^M - 1\}$ (e.g. 0–255 for 8 bits).
- **Signed** integers use the most significant bit as a sign bit and **two's complement** for negatives: to represent $-x$, flip all bits of x and add 1. Equivalently, the bit pattern of $-x$ is that of the unsigned integer $2^M - x$. The range is $[-2^{M-1}, 2^{M-1} - 1]$; for $M = 32$ the maximum is $2^{31} - 1 = 2147483647$.

Two's complement turns subtraction into addition modulo 2^M, correct whenever the result is in range: with 8 bits, $57 - 35$ is $00111001_2 + 11011101_2 \equiv 00010110_2 = 22$.

Out-of-range results **overflow**: C and Julia wrap around ($255 + 1 = 0$ for unsigned 8-bit integers), R returns `NA`. Arbitrary-precision integers avoid overflow at a cost in speed.

1.3 Floating-point numbers

A real number is written in scientific notation with base $b = 2$:

$$\pm d_1.d_2 \cdots d_p \times 2^e, \qquad d_i \in \{0, 1\}.$$

In **normalized** form $d_1 = 1$ (e.g. $18 = 1.0010_2 \times 2^4$), so the leading bit is not stored. Exactly representable numbers are **floating-point numbers**; other reals are rounded to a nearby one. IEEE 754 (1985) stores

- a sign bit,
- the fraction (mantissa, significand) of the normalized representation, $p - 1$ bits,
- the exponent plus a **bias**.

format	bits	fraction bits	p	$e_{\min}, e_{\max}$	range	decimal digits
half (Float16)	16	10	11	$-14, 15$	$10^{\pm 4}$	3.3
single (float)	32	23	24	$-126, 127$	$10^{\pm 38}$	7.2
double	64	52	53	$-1022, 1023$	$10^{\pm 308}$	15.9

The bias is 15, 127 and 1023 respectively; the precision in decimal digits is $\log_{10} 2^p$. For half precision, the value of the bit string $b_{15}b_{14}\cdots b_0$ is

$$(-1)^{b_{15}} \times 2^{(b_{14}\cdots b_{10})_2 - 15} \times \Big(1 + \sum_{i=1}^{10} b_{10-i} 2^{-i}\Big).$$

Double precision is the default for numerical computing (R has no native single precision).

Special values. The exponent codes $e_{\min}-1$ (all zeros) and $e_{\max}+1$ (all ones) are reserved.

- Exponent $e_{\max}+1$ with zero fraction: $\pm\infty$ (e.g. $10^{300}\times 10^{300}$).
- Exponent $e_{\max}+1$ with nonzero fraction: `NaN` (e.g. $0/0$, $0\cdot\infty$). A `NaN` is not equal to anything, including itself.
- Exponent $e_{\min}-1$ with zero fraction: an exact zero (which cannot be written in normalized form).
- Exponent $e_{\min}-1$ with nonzero fraction: **denormalized** (subnormal) numbers $0.d_2\cdots d_p\times 2^{e_{\min}}$ filling $(0, 2^{e_{\min}})$. This **gradual underflow** guarantees $x-y=0\Rightarrow x=y$; e.g. $2^{-149}=0.0\cdots 01_2\times 2^{-126}$ in single precision.

1.4 Rounding and machine epsilon

A real number with more than p significant bits is rounded. IEEE 754's default is **round to nearest, ties to even** (at a tie, the neighbour whose last bit is 0). With $p=4$:

exact	rounded	reason
1.000011×2^1	1.000×2^1	remainder $011<100$: down
1.000110×2^1	1.001×2^1	remainder $110>100$: up
1.011100×2^1	1.100×2^1	tie: to even (up)
1.010100×2^1	1.010×2^1	tie: to even (down)

If x is represented by $[x]$, the **absolute error** is $|[x]-x|$ and the **relative error** is $|[x]-x|/|x|$.

Floating-point numbers are not evenly spaced: each $(2^i, 2^{i+1}]$ holds 2^{p-1} equally spaced ones, so the spacing doubles per binade.

Definition (Machine epsilons)

$\epsilon_{\max}$ is the smallest positive floating-point number whose addition to 1 gives a result different from 1; $\epsilon_{\min}$ is the smallest positive floating-point number whose subtraction from 1 gives a result different from 1.

Proposition 1.1

With precision p and ties to even,

$$\epsilon_{\max}=2^{-p}+2^{-(2p-1)},\qquad \epsilon_{\min}=2^{-(p+1)}+2^{-2p}.$$

For double precision, $\epsilon_{\max}=2^{-53}+2^{-105}\approx 1.1\times 10^{-16}$.

Proof. The neighbours of 1 are $1+2^{-(p-1)}$ and $1-2^{-p}$. The midpoint $1+2^{-p}$ ties to the even value 1, so $\epsilon_{\max}>2^{-p}$. The next floating-point number after 2^{-p} is $2^{-p}+2^{-p-(p-1)}$; adding it to 1 passes the midpoint and rounds to $1+2^{-(p-1)}$, so $\epsilon_{\max}=2^{-p}+2^{-(2p-1)}$. Likewise $1-2^{-(p+1)}$, the midpoint of $1-2^{-p}$ and 1, rounds to 1, while the next number $2^{-(p+1)}+2^{-(p+1)-(p-1)}=2^{-(p+1)}+2^{-2p}$ does move 1. ■

Theorem 1.2 (Relative error of representation)

If a real number $x > 0$ in the normal range is represented by $[x]$, then

$$\frac{|[x] - x|}{|x|} \leq \epsilon_{\max}.$$

Proof. Let $[x] = (1 + \sum_{i=1}^{p-1} b_{i+1} 2^{-i}) \times 2^e$, so $[x] \geq 2^e$ and the spacing of floating-point numbers near $[x]$ is $2^{e-(p-1)}$. Rounding to nearest gives $|x - [x]| \leq 2^e/2^p$. Then

$$\frac{|x - [x]|}{|x|} \leq \frac{2^e}{2^p} \cdot \frac{1}{[x] - 2^e/2^p} \leq \frac{2^e}{2^p} \cdot \frac{1}{2^e(1 - 2^{-p})} \leq \frac{1}{2^p}\left(1 + \frac{1}{2^{p-1}}\right) = \epsilon_{\max},$$

using $1/(1-u) \leq 1 + 2u$ for $0 \leq u \leq \frac{1}{2}$. ■

So $[x] = x(1 + \delta)$ with $|\delta| \leq \epsilon_{\max}$: the machine epsilon is the **unit roundoff** or machine precision.

Example 1.1 (A toy decimal system)

Take $b = 10$, $p = 5$, $-9 \leq e \leq 9$, numbers $\pm 0.d_1 \cdots d_5 \times 10^e$, normalized except that subnormals and zero are allowed at $e = -9$. Let $X \sim \text{Unif}(0, 1)$ and round to nearest.

- The smallest positive number is $0.00001 \times 10^{-9} = 10^{-14}$, so $[X] = 0$ iff $X \leq \frac{1}{2} \cdot 10^{-14}$ (ties to the even neighbour 0): $P([X] = 0) = 5 \times 10^{-15}$.
- $1 = 0.10000 \times 10^1$ and its predecessor is 0.99999×10^0, so $[X] = 1$ iff $X \geq 0.999995$: $P([X] = 1) = 5 \times 10^{-6}$.
- $[X]$ is not uniform over the floating-point numbers in $(0, 1)$: within $[10^{e-1}, 10^e)$ the spacing is 10^{e-5}, so each number there has probability 10^{e-5}. For instance $P([X] = 0.5) = 10^{-5}$ but $P([X] = 0.05) = 10^{-6}$.

1.5 Floating-point arithmetic and its pitfalls

IEEE 754 computes $+, -, \times, \div, \sqrt{\ }$, remainder and conversions *as if exactly* and then rounds: the computed sum of $[x]$ and $[y]$ is $[[x] + [y]]$.

Overflow and underflow. Magnitudes beyond $\approx 10^{308}$ overflow to $\pm\infty$; below $\approx 10^{-308}$ they underflow gradually to subnormals, then 0. Underflow is the lesser evil, so arrange formulas to avoid overflow, e.g.

$$p = \frac{1}{1 + e^{-x^\top \beta}} \qquad \text{rather than} \qquad \frac{e^{x^\top \beta}}{1 + e^{x^\top \beta}},$$

since the latter gives $\infty/\infty =$ `NaN` for large $x^\top \beta$ while the former only underflows. Likewise $\log(e^{710}) = \infty$: work with likelihoods on the log scale.

Adding numbers of different magnitudes. If $a = x.xx \cdots \times 2^{30}$ and $b = y.yy \cdots \times 2^{-30}$, then b lies below the last bit of a and $a + b = a$ in double precision.

Proposition 1.3 (Error of a sum)

For $x, y > 0$, the computed sum satisfies

$$\frac{|[[x] + [y]] - (x + y)|}{x + y} \leq 2\epsilon_{\max} + \epsilon_{\max}^2 \approx 2\epsilon_{\max}.$$

Proof. Write $[x] = x(1+\delta_x)$, $[y] = y(1+\delta_y)$ and $[[x]+[y]] = ([x]+[y])(1+\delta_s)$, all $|\delta| \le \epsilon_{\max}$. Then $[x]+[y] \le (x+y)(1+\epsilon_{\max})$ and

$$|[[x]+[y]] - (x+y)| \le |\delta_s|([x]+[y]) + |\delta_x|x + |\delta_y|y \le \epsilon_{\max}(1+\epsilon_{\max})(x+y) + \epsilon_{\max}(x+y).$$

■

Adding positive numbers is benign; subtraction is not.

Catastrophic cancellation. Subtracting nearly equal numbers that carry rounding errors destroys significant digits. With 3 digits, $b = 3.34$, $a = 1.22$, $c = 2.28$: exactly $b^2 - 4ac = 0.0292$, but $[b^2] = 11.2$, $[4ac] = 11.1$ give 0.1; the subtraction is exact and magnifies earlier errors. In binary, $[x] = 1.d_2 \cdots d_{p-1}1$ and $[y] = 1.d_2 \cdots d_{p-1}0$ give $[x] - [y] = 2^{-(p-1)}$, while $x - y$ may be anywhere in $(0, 2^{-(p-2)} + 2^{-2p}]$: the relative error is unbounded. Likewise $1 - \Phi(z)$ returns 0 for $z \ge 9$; compute tail probabilities directly.

Rules of thumb.

(1) Add small numbers before large ones.

(2) Add numbers of like magnitude together (pairwise summation).

(3) Avoid subtracting nearly equal numbers. For example, for $ax^2 + bx + c = 0$ with $b > 0$, compute $x_1 = \frac{-b-\sqrt{b^2-4ac}}{2a}$ (no cancellation) and then $x_2 = c/(ax_1)$ from $x_1x_2 = c/a$, instead of $\frac{-b+\sqrt{b^2-4ac}}{2a}$.

Floating-point arithmetic breaks algebraic laws: addition is not associative ($(a+b)-a \ne b$ for $a \gg b$), multiplication does not distribute exactly, zero has two signs ± 0 (equal under comparison), $x + 0 = x$ fails for `NaN`, and $x - x = 0$ fails for $\pm\infty$ and `NaN`. Decimal constants such as $0.1, 0.3$ are inexact, so `0.1+0.2==0.3` and `0.1==0.3/3` are false; test equality with a tolerance (R's `all.equal`).

Example 1.2 (Cancellation in the quadratic formula)

For $a = 10^{-30}$, $b = -10^{30}$, $c = 10^{30}$ the roots are about 10^{60} and 1. In double precision $4ac = 4$ is below the last bit of $b^2 = 10^{60}$, so $\sqrt{b^2 - 4ac} = |b|$ and the textbook formula gives the small root $(-b - |b|)/(2a) = 0$. The stable recipe

$$q = -\tfrac{1}{2}(b + \operatorname{sign}(b)\sqrt{b^2 - 4ac}), \qquad x_1 = q/a, \qquad x_2 = c/q,$$

never subtracts nearly equal numbers and gives $x_2 = 1$ to full precision (for $b = +10^{30}$, reverse the signs).

Example 1.3 (The harmonic series "converges")

The partial sums $s_n \approx \log n$ diverge, but in floating point they freeze once $1/n$ is below half the spacing near s_n: in single precision at $n = 2^{21} = 2097152$ with $s = 15.4037$; in double precision (spacing 2^{-47} on $[32, 64)$) near 34 after about $2^{48} \approx 3 \times 10^{14}$ terms.
For Euler's constant $\gamma = \lim_n(\sum_{i \le n} 1/i - \log n) \approx 0.5772$: take $n = 10^6$, add $1/n + \cdots + 1/1$ (small first), subtract $\log n$ and the leading error term $1/(2n)$. This gives about 13 digits; without the correction the error is 5×10^{-7}.

1.6 Selected exercises

Motivation. Three facts used above: the $1/(2n)$ correction for γ, exact floating-point subtraction, and error growth in long sums.

Exercise 1.1 (Euler's constant as an integral)

Let $\lfloor x \rfloor$ be the largest integer not exceeding x, and $H_n = \sum_{i=1}^{n} 1/i$.
(a) Show that for every integer $n \geq 1$,

$$H_n - \log n = \frac{1}{n} + \int_1^n \Big(\frac{1}{\lfloor x \rfloor} - \frac{1}{x}\Big) dx.$$

(b) Deduce that $\gamma = \lim_n (H_n - \log n)$ exists, that $\gamma = \int_1^\infty (1/\lfloor x \rfloor - 1/x)\, dx$, that $0 < \gamma < 1$, and that $0 < H_n - \log n - \gamma < 1/n$.
(c) Show that $H_n - \log n - \gamma = \frac{1}{2n} + O(n^{-2})$, which justifies the correction used in Example 1.3.

Solution. Write $r(x) = 1/\lfloor x \rfloor - 1/x$, which is ≥ 0 because $\lfloor x \rfloor \leq x$, and $c_i = \int_i^{i+1} r(x)\, dx$ for integers $i \geq 1$.

(a) On $[i, i+1)$ we have $\lfloor x \rfloor = i$, so

$$c_i = \frac{1}{i} - \log \frac{i+1}{i}.$$

Summing over $i = 1, \ldots, n-1$, the logarithms telescope: $\int_1^n r = H_{n-1} - \log n = H_n - \frac{1}{n} - \log n$.

(b) On $[i, i+1]$, $1/x \geq 1/(i+1)$, so $0 < c_i \leq \frac{1}{i} - \frac{1}{i+1}$ ($r > 0$ on $(i, i+1)$). Hence

$$0 < \int_n^\infty r = \sum_{i \geq n} c_i \leq \sum_{i \geq n} \Big(\frac{1}{i} - \frac{1}{i+1}\Big) = \frac{1}{n}.$$

So $\int_1^\infty r < \infty$, and by (a) $H_n - \log n = \frac{1}{n} + \int_1^\infty r - \int_n^\infty r \to \int_1^\infty r = \gamma$. Also $H_n - \log n - \gamma = \frac{1}{n} - \int_n^\infty r \in (0, \frac{1}{n})$, positive because $c_i < \frac{1}{i} - \frac{1}{i+1}$ strictly ($1/x > 1/(i+1)$ on $(i, i+1)$). Finally $\gamma > 0$ as $r > 0$ on $(1, 2)$, and $\gamma < 1$ by the display with $n = 1$ and strictness.

(c) For $0 < t \leq 1$ the series $\log(1+t) = t - \frac{t^2}{2} + \frac{t^3}{3} - \ldots$ is alternating with decreasing terms, so $t - \frac{t^2}{2} \leq \log(1+t) \leq t - \frac{t^2}{2} + \frac{t^3}{3}$. With $t = 1/i$,

$$\frac{1}{2i^2} - \frac{1}{3i^3} \leq c_i \leq \frac{1}{2i^2}.$$

Comparing sums with integrals, $\frac{1}{n} = \int_n^\infty x^{-2} dx \leq \sum_{i \geq n} i^{-2} \leq \frac{1}{n^2} + \frac{1}{n}$, and $\sum_{i \geq n} i^{-3} \leq \frac{1}{n^3} + \frac{1}{2n^2}$. Therefore $\int_n^\infty r = \sum_{i \geq n} c_i = \frac{1}{2n} + O(n^{-2})$, and by (b)

$$H_n - \log n - \gamma = \frac{1}{n} - \int_n^\infty r = \frac{1}{2n} + O(n^{-2}).$$

Subtracting $1/(2n)$ reduces the error from $O(n^{-1})$ to $O(n^{-2})$. ■

Exercise 1.2 (Sterbenz's lemma: exact subtraction)

Consider binary floating-point numbers with precision p, exponent range $[e_{\min}, e_{\max}]$ and gradual underflow, and ignore overflow.
(a) Show that if x, y are positive floating-point numbers with $y/2 \leq x \leq 2y$, then $x - y$ is a floating-point number; hence the computed difference $[x - y]$ is exact.

(b) Show by an example that the hypothesis cannot be dropped.

Solution. [Supplement]

Step 1. (Representability.) Every $D \cdot 2^{f-p+1}$ with integers $0 \le D < 2^p$ and $f \in [e_{\min}, e_{\max}]$ is a floating-point number. $D = 0$ is the exact zero. Otherwise let $2^k \le D < 2^{k+1}$ ($k \le p-1$) and write $D \cdot 2^{f-p+1} = (D/2^k) \cdot 2^{f'}$, $f' = f - (p-1-k) \le e_{\max}$, where $D/2^k \in [1, 2)$ has at most p significant bits. If $f' \ge e_{\min}$ it is normalized; if $f' < e_{\min}$ it is a multiple of $2^{e_{\min}-p+1}$ (as $f \ge e_{\min}$) in $(0, 2^{e_{\min}})$, i.e. subnormal.

Step 2. (Common grid.) By symmetry ($x - y = -(y - x)$) assume $x \ge y$. With e_y the exponent of y ($e_y = e_{\min}$ if y is subnormal), $y = M_y 2^{e_y-p+1}$ with an integer $0 < M_y < 2^p$. As $e_x \ge e_y$, $x = M_x 2^{e_x-e_y} \cdot 2^{e_y-p+1}$ is also an integer multiple of 2^{e_y-p+1}.

Step 3. (Conclusion.) So $x - y = D \cdot 2^{e_y-p+1}$ with an integer $D \ge 0$, and $x \le 2y$ gives $x - y \le y$, i.e. $D \le M_y < 2^p$. By Step 1 ($f = e_y$), $x - y$ is a floating-point number; rounding returns it unchanged, so $[x - y] = x - y$.

(b) $x = 2^p$, $y = \frac{1}{2}$ (so $x > 2y$): $x - y = (2^{p+1} - 1)/2$ is $p + 1$ ones in binary, needs $p + 1$ significant bits, and is not representable. ■

Exercise 1.3 (Rounding error of recursive summation)

Let $x_1, \ldots, x_n$ be floating-point numbers, $u = \epsilon_{\max}$, and suppose no overflow or underflow occurs. The loop $\hat{s}_1 = x_1$, $\hat{s}_k = [\hat{s}_{k-1} + x_k]$ computes $\hat{s}_n$.
(a) Show that

$$\Big|\hat{s}_n - \sum_{i=1}^{n} x_i\Big| \le \gamma_{n-1} \sum_{i=1}^{n} |x_i|, \qquad \gamma_m = (1+u)^m - 1 \le \frac{mu}{1-mu} \quad (mu < 1).$$

(b) Show the sharper bound $|\hat{s}_n - \sum_i x_i| \le \sum_{i=1}^{n} \gamma_{n+1-\max(i,2)} |x_i|$, and explain why it supports adding numbers of small magnitude first.

Solution. [Supplement]

Step 1. (One addition.) By Theorem 1.2, $\hat{s}_k = (\hat{s}_{k-1} + x_k)(1 + \delta_k)$ with $|\delta_k| \le u$ (take $\delta_k = 0$ if the exact sum is 0, which is representable).

Step 2. (Unrolling.) By induction on n,

$$\hat{s}_n = \sum_{i=1}^{n} x_i \prod_{k=\max(i,2)}^{n} (1 + \delta_k),$$

since multiplying the formula for $\hat{s}_{n-1}$ by $(1+\delta_n)$ and adding $x_n(1+\delta_n)$ gives it for $\hat{s}_n$. So x_i carries $m_i = n + 1 - \max(i, 2) \le n - 1$ factors.

Step 3. (Products of rounding factors.) For $|\delta_k| \le u$, expanding the product,

$$\Big|\prod_{k=1}^{m}(1+\delta_k) - 1\Big| = \Big|\sum_{\emptyset \ne S \subseteq \{1,\ldots,m\}} \prod_{k \in S} \delta_k\Big| \le \sum_{\emptyset \ne S} u^{|S|} = (1+u)^m - 1 = \gamma_m.$$

Step 4. (Bounds.) Subtracting $\sum_i x_i$ in Step 2 and using Step 3 gives (b): $|\hat{s}_n - \sum_i x_i| \le \sum_i \gamma_{m_i} |x_i|$. Since γ_m increases in m and $m_i \le n - 1$, (a) follows. For the bound on γ_m: $(1+u)^m \le e^{mu}$, and $e^{-mu} \ge 1 - mu > 0$ gives $e^{mu} \le 1/(1-mu)$, so $\gamma_m \le \frac{1}{1-mu} - 1 = \frac{mu}{1-mu}$.

In (b), x_1, x_2 carry γ_{n-1} and x_n only γ_1; by the rearrangement inequality the bound is smallest when the $|x_i|$ increase, i.e. small terms first (rule (1)). For positive x_i the relative error is at most $\gamma_{n-1} \approx (n-1)u$, consistent with Proposition 1.3 for $n = 2$. ■